The plan for fine-tuning RoBERTa with the UCC

1. Set up

- Install transformers and pytorch lightning and import required packages.
- Loads UCC data and inspect it.

2. Create a dataset

- Load data into pytorch dataset

3. Build the model

- Create a new pytorch (lightning) model with the Hugging Face RoBERTa model and a multi-label classification head.

4. Test model performance

- Test model performance using the ROC AUC and same test-set used in the original paper.

```
#!nvidia-smi
```

Imports

```
%%capture
!pip install transformers
!pip install pytorch-lightning
```

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import drive
```

```
drive.mount('/content/drive', force_remount=False)
```

```
Mounted at /content/drive
```

```
train_path = '/content/drive/MyDrive/ucc/ucc_train.csv'
val_path = '/content/drive/MyDrive/ucc/ucc_val.csv'
```

```
train_data = pd.read_csv(train_path)
```

```
train_data.head(5)
```

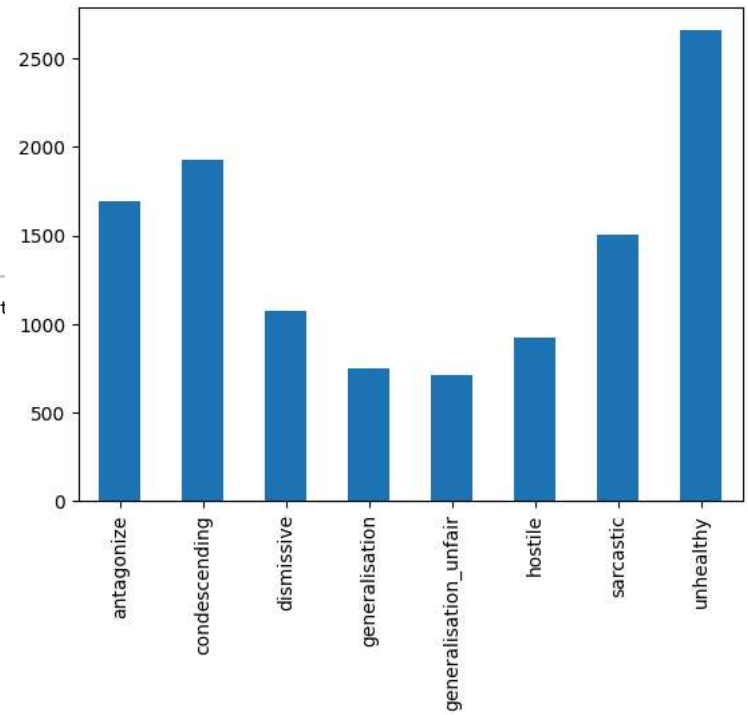
inspect data

```
train_data['unhealthy'] = np.where(train_data['healthy'] == 1, 0, 1)

attributes = ['antagonize', 'condescending','dismissive','generalisation',
              'generalisation_unfair','hostile','sarcastic','unhealthy']

train_data[attributes].sum().plot.bar()
```

```
<Axes: >
```



Next

Dataset

```
from torch.utils.data import Dataset
```

```
class UCC_Dataset(Dataset):

  def __init__(self, data_path, tokenizer, attributes, max_token_len: int = 128, sample = 5000):
    self.data_path = data_path
    self.tokenizer = tokenizer
    self.attributes = attributes
    self.max_token_len = max_token_len
    self.sample = sample
    self._prepare_data()


  def _prepare_data(self):
    data = pd.read_csv(self.data_path)
    data['unhealthy'] = np.where(data['healthy'] == 1, 0, 1)
    if self.sample is not None:
      unhealthy = data.loc[data[attributes].sum(axis=1) > 0]
      clean = data.loc[data[attributes].sum(axis=1) == 0]
      self.data = pd.concat([unhealthy, clean.sample(self.sample, random_state=7)])
    else:
      self.data = data

  def __len__(self):
```

```
    return len(self.data)

  def __getitem__(self, index):
    item = self.data.iloc[index]
    comment = str(item.comment)
    attributes = torch.FloatTensor(item[self.attributes])
    tokens = self.tokenizer.encode_plus(comment,
                                        add_special_tokens=True,
                                        return_tensors='pt',
                                        truncation=True,
                                        padding='max_length',
                                        max_length=self.max_token_len,
                                        return_attention_mask = True)
    return {'input_ids': tokens.input_ids.flatten(), 'attention_mask': tokens.attention_mask.flatten(), 'labels': attribute
```

```
import torch
from transformers import AutoTokenizer
model_name = 'roberta-base'
tokenizer = AutoTokenizer.from_pretrained(model_name)
ucc_ds = UCC_Dataset(train_path, tokenizer, attributes=attributes)
ucc_ds_val = UCC_Dataset(val_path, tokenizer, attributes=attributes, sample=None)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100%  25.0/25.0 [00:00<00:00, 1.78kB/s]
config.json: 100%  481/481 [00:00<00:00, 56.8kB/s]
vocab.json: 100%  899k/899k [00:00<00:00, 6.65MB/s]
merges.txt: 100%  456k/456k [00:00<00:00, 3.73MB/s]
tokenizer.json: 100%  1.36M/1.36M [00:00<00:00, 6.51MB/s]
```

```
ucc_ds.__getitem__(0)['labels'].shape, ucc_ds.__getitem__(0)['input_ids'].shape, ucc_ds.__getitem__(0)['attention_mask'].sh
#ucc_ds.__getitem__(0)
```

```
/tmp/ipython-input-3005120282.py:28: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future
  attributes = torch.FloatTensor(item[self.attributes])
/tmp/ipython-input-3005120282.py:28: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future
  attributes = torch.FloatTensor(item[self.attributes])
/tmp/ipython-input-3005120282.py:28: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future
  attributes = torch.FloatTensor(item[self.attributes])
(torch.Size([8]), torch.Size([128]), torch.Size([128]))
```

```
len(ucc_ds)
```

```
9960
```

```
len(ucc_ds_val)
```

```
4427
```

Data module

```
import pytorch_lightning as pl
from torch.utils.data import DataLoader
```

```
class UCC_Data_Module(pl.LightningDataModule):

  def __init__(self, train_path, val_path, attributes, batch_size: int = 16, max_token_length: int = 128,  model_name='robe
    super().__init__()
    self.train_path = train_path
    self.val_path = val_path
    self.attributes = attributes
    self.batch_size = batch_size
    self.max_token_length = max_token_length
    self.model_name = model_name
    self.tokenizer = AutoTokenizer.from_pretrained(model_name)

  def setup(self, stage = None):
    if stage in (None, "fit"):
```

```
      self.train_dataset = UCC_Dataset(self.train_path, attributes=self.attributes, tokenizer=self.tokenizer)
      self.val_dataset = UCC_Dataset(self.val_path, attributes=self.attributes, tokenizer=self.tokenizer, sample=None)
    if stage == 'predict':
      self.val_dataset = UCC_Dataset(self.val_path, attributes=self.attributes, tokenizer=self.tokenizer, sample=None)

  def train_dataloader(self):
    return DataLoader(self.train_dataset, batch_size = self.batch_size, num_workers=4, shuffle=True)

  def val_dataloader(self):
    return DataLoader(self.val_dataset, batch_size = self.batch_size, num_workers=4, shuffle=False)

  def predict_dataloader(self):
    return DataLoader(self.val_dataset, batch_size = self.batch_size, num_workers=4, shuffle=False)
```

```
ucc_data_module = UCC_Data_Module(train_path, val_path, attributes=attributes)
```

```
ucc_data_module.setup()
```

```
ucc_data_module.train_dataloader()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:627: UserWarning: This DataLoader will create 4 worke
  warnings.warn(
<torch.utils.data.dataloader.DataLoader at 0x7d17475faf90>
```

```
len(ucc_data_module.train_dataloader())
```

```
623
```

Model

```
from transformers import AutoModel, get_cosine_schedule_with_warmup
import torch.nn as nn
import math
from torchmetrics.functional.classification import auroc
import torch.nn.functional as F
from torch.optim import AdamW
```

```
class UCC_Comment_Classifier(pl.LightningModule):

  def __init__(self, config: dict):
    super().__init__()
    self.config = config
    self.pretrained_model = AutoModel.from_pretrained(config['model_name'], return_dict = True)
    self.hidden = torch.nn.Linear(self.pretrained_model.config.hidden_size, self.pretrained_model.config.hidden_size)
    self.classifier = torch.nn.Linear(self.pretrained_model.config.hidden_size, self.config['n_labels'])
    torch.nn.init.xavier_uniform_(self.classifier.weight)
    self.loss_func = nn.BCEWithLogitsLoss(reduction='mean')
    self.dropout = nn.Dropout()

  def forward(self, input_ids, attention_mask, labels=None):
    # roberta layer
    output = self.pretrained_model(input_ids=input_ids, attention_mask=attention_mask)
    pooled_output = torch.mean(output.last_hidden_state, 1)
    # final logits
    pooled_output = self.dropout(pooled_output)
    pooled_output = self.hidden(pooled_output)
    pooled_output = F.relu(pooled_output)
    pooled_output = self.dropout(pooled_output)
    logits = self.classifier(pooled_output)
    # calculate loss
    loss = 0
    if labels is not None:
      loss = self.loss_func(logits.view(-1, self.config['n_labels']), labels.view(-1, self.config['n_labels']))
    return loss, logits

  def training_step(self, batch, batch_index):
    loss, outputs = self(**batch)
    self.log("train loss ", loss, prog_bar = True, logger=True)
    return {"loss":loss, "predictions":outputs, "labels": batch["labels"]}

  def validation_step(self, batch, batch_index):
    loss, outputs = self(**batch)
    self.log("validation loss ", loss, prog_bar = True, logger=True)
    return {"val_loss": loss, "predictions":outputs, "labels": batch["labels"]}
```

```
  def predict_step(self, batch, batch_index):
    loss, outputs = self(**batch)
    return outputs

  def configure_optimizers(self):
    optimizer = AdamW(self.parameters(), lr=self.config['lr'], weight_decay=self.config['weight_decay'])
    total_steps = self.config['train_size']/self.config['batch_size']
    warmup_steps = math.floor(total_steps * self.config['warmup'])
    warmup_steps = math.floor(total_steps * self.config['warmup'])
    scheduler = get_cosine_schedule_with_warmup(optimizer, warmup_steps, total_steps)
    return [optimizer],[scheduler]

  # def validation_epoch_end(self, outputs):
  #   losses = []
  #   for output in outputs:
  #     loss = output['val_loss'].detach().cpu()
  #     losses.append(loss)
  #   avg_loss = torch.mean(torch.stack(losses))
  #   self.log("avg_val_loss", avg_loss)
```

```
config = {
    'model_name': 'distilroberta-base',
    'n_labels': len(attributes),
    'batch_size': 128,
    'lr': 1.5e-6,
    'warmup': 0.2,
    'train_size': len(ucc_data_module.train_dataloader()),
    'weight_decay': 0.001,
    'n_epochs': 10
}

model = UCC_Comment_Classifier(config)
```

```
config.json: 100%  480/480 [00:00<00:00, 42.4kB/s]
model.safetensors: 100%  331M/331M [00:05<00:00, 47.1MB/s]
```

```
idx=0
input_ids = ucc_ds.__getitem__(idx)['input_ids']
attention_mask = ucc_ds.__getitem__(idx)['attention_mask']
labels = ucc_ds.__getitem__(idx)['labels']
model.cpu()
loss, output = model(input_ids.unsqueeze(dim=0), attention_mask.unsqueeze(dim=0), labels.unsqueeze(dim=0))
print(labels.shape, output.shape, output)
```

```
/tmp/ipython-input-3005120282.py:28: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future
  attributes = torch.FloatTensor(item[self.attributes])
/tmp/ipython-input-3005120282.py:28: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future
  attributes = torch.FloatTensor(item[self.attributes])
/tmp/ipython-input-3005120282.py:28: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future
  attributes = torch.FloatTensor(item[self.attributes])
torch.Size([8]) torch.Size([1, 8]) tensor([[ 0.6356, -0.1083,  0.0579, -0.8492,  1.0693, -1.5794,  0.6147, -0.9176]],
       grad_fn=<AddmmBackward0>)
```

train model

```
# datamodule
ucc_data_module = UCC_Data_Module(train_path, val_path, attributes=attributes, batch_size=config['batch_size'])
ucc_data_module.setup()

# model
model = UCC_Comment_Classifier(config)

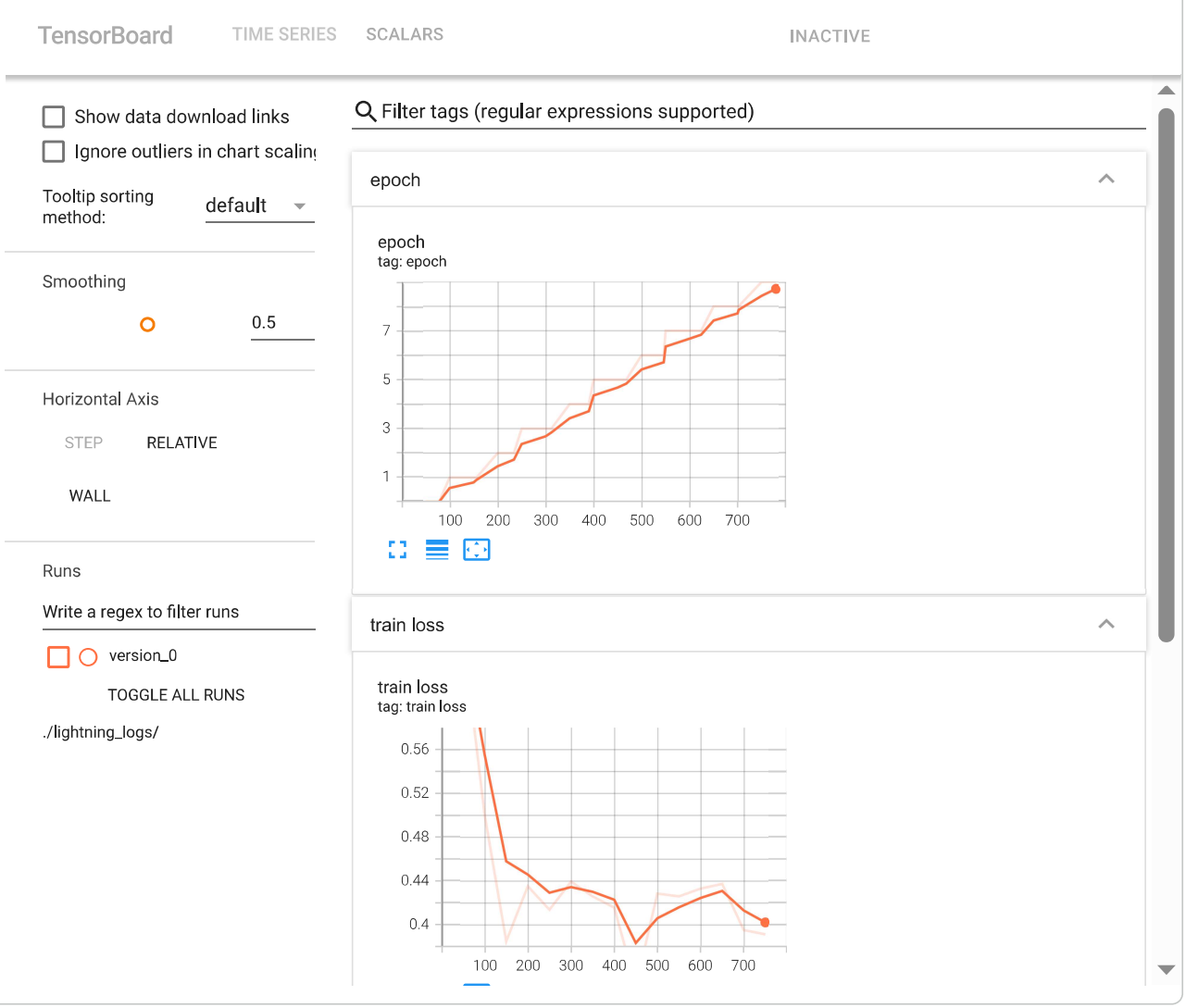
# trainer and fit
trainer = pl.Trainer(max_epochs=config['n_epochs'],
                     #accelerator="gpu",
                     #devices=1,
                     num_sanity_val_steps=50)
trainer.fit(model, ucc_data_module)
```

```
INFO:pytorch_lightning.utilities.rank_zero:💡 Tip: For seamless cloud uploads and versioning, try installing [litmodels](htt
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
```

	Name	Type	Params	Mode	FLOPs
0	pretrained_model	RobertaModel	82.1 M	eval	0
1	hidden	Linear	590 K	train	0
2	classifier	Linear	6.2 K	train	0
3	loss_func	BCEWithLogitsLoss	0	train	0
4	dropout	Dropout	0	train	0

```
Trainable params: 82.7 M
Non-trainable params: 0
Total params: 82.7 M
Total estimated model params size (MB): 330
Modules in train mode: 4
Modules in eval mode: 120
Total FLOPs: 0
Epoch 9/9 ━━━━━━━━━━━━━━━━━━━━ 78/78 0:01:31 • 0:00:00 0.87it/s v_num: 0.000 train loss : 0.368
                                                                validation loss : 0.208
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:627: UserWarning: This DataLoader will
create 4 worker processes in total. Our suggested max number of worker in current system is 2, which is smaller
than what this DataLoader is going to create. Please be aware that excessive worker creation might get DataLoader
running slow or even freeze, lower the worker number to avoid potential slowness/freeze if necessary.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/pytorch_lightning/loops/fit_loop.py:534: Found 120 module(s) in eval mode
at the start of training. This may lead to unexpected behavior during training. If this is intentional, you can
ignore this warning.
```

```
%load_ext tensorboard
%tensorboard --logdir ./lightning_logs/
```



Predict with model

```
# method to convert list of comments into predictions for each comment
def classify_raw_comments(model, dm):
```

```
  predictions = trainer.predict(model, datamodule=dm)
  flattened_predictions = np.stack([torch.sigmoid(torch.Tensor(p)) for batch in predictions for p in batch])
  return flattened_predictions
```

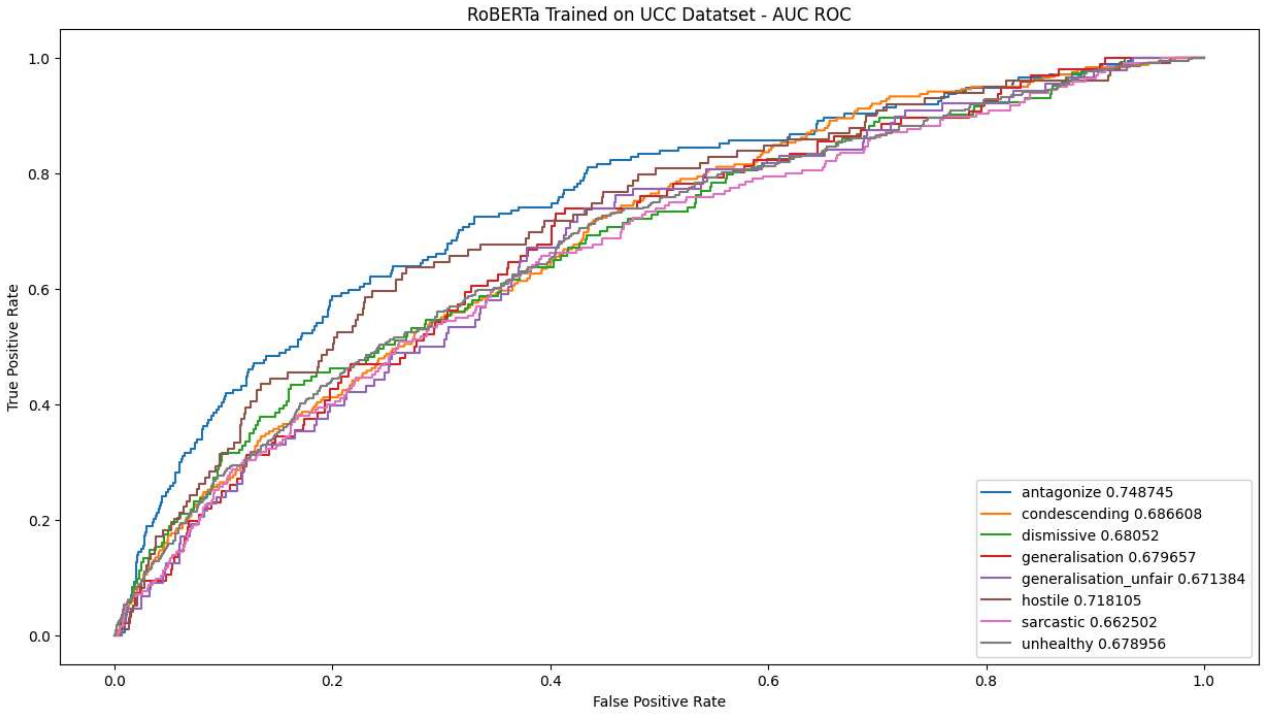
```
predictions = classify_raw_comments(model, ucc_data_module)
```

Show hidden output

```
val_data = pd.read_csv(val_path)
val_data['unhealthy'] = np.where(val_data['healthy'] == 1, 0, 1)
true_labels = np.array(val_data[attributes])
```

```
from sklearn import metrics
plt.figure(figsize=(15, 8))
for i, attribute in enumerate(attributes):
  fpr, tpr, _ = metrics.roc_curve(
      true_labels[:,i].astype(int), predictions[:, i])
  auc = metrics.roc_auc_score(
      true_labels[:,i].astype(int), predictions[:, i])
  plt.plot(fpr, tpr, label='%s %g' % (attribute, auc))
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc='lower right')
plt.title('RoBERTa Trained on UCC Datatset - AUC ROC')
```

```
Text(0.5, 1.0, 'RoBERTa Trained on UCC Datatset - AUC ROC')
```



Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```